sustainable at scale. The patterns that worked for C³ apply to nearly any system using a policy engine.

13.7.1 Key insights

The C³ pilot gave ACME a laboratory to test ideas that initially seemed abstract on paper. By the time the system stabilized, a few insights had surfaced, patterns that transformed early lessons into lasting architectural practices.

- **Separation brings clarity**. Splitting authorization into separate layers —system, application, and tenant—made ownership and troubleshooting much simpler. Each layer enforces policies suitable for its trust boundary.
- **Identity is the foundation**. Federated OIDC and a consistent tenant model provided C³ with a reliable anchor for every policy decision. Without clear identity boundaries, even elegant policies fail.
- **Context needs to stay current**. Device posture, risk scores, and other dynamic signals quickly become outdated. The PEP's role is not only enforcement but also assembling context.
- **UI visibility is not enforcement**. The interface may use graph queries and partial evaluation to display what a person could access, but the backend still makes the final decision.
- **Templates make customization safe**. Providing tenants with structured policy templates offers flexibility without losing control. Guardrails are as important as expressiveness.

Together, these insights reveal that authorization isn't just a subsystem—it's an ecosystem. When policy, identity, and context develop together, externalized authorization *shifts from being merely a security requirement to a strategic advantage*.

13.7.2 Integration checklist

Teams planning to integrate a policy engine into an existing system can use the following steps as a guide. Table 13.4 summarizes the sequence ACME followed when building the Customer Collaboration Cloud.